Method and System for Communication via Passive Reflection from Non-Cooperative Orbiting Objects

Reinaldo J. Aguiar Marcano

March 7, 2026

Abstract

A method and system for establishing a communication link between two points on or near the Earth's surface by using non-cooperative orbiting objects, such as satellites or other space debris, as passive reflectors for a directed radio signal. The system does not require any active participation or modification of the orbiting objects. A ground-based system calculates the future position of a suitable object that will momentarily align to reflect a signal from a transmitter to a receiver. By precisely timing the transmission to coincide with the object's passage through the reflection point, a transient communication link is created. The invention provides a means for long-distance, over-the-horizon communication that is difficult to intercept and does not rely on traditional geostationary communication satellites or terrestrial infrastructure. The system can operate with either movable, directable antennas or an array of fixed-orientation antennas.

1 Background of the Invention

Over-the-horizon communication has traditionally relied on either bouncing signals off the ionosphere, using dedicated communication satellites, or laying extensive terrestrial infrastructure like fiber optic cables. Dedicated satellites are expensive to build and launch, and their communication channels are often monitored and controlled. Terrestrial infrastructure is vulnerable to physical disruption and is not available in many remote or oceanic areas.

While techniques like tropospheric scatter exist for point-to-point military communication, they are limited in range and require specific atmospheric conditions. There exists a need for a communication method that can bridge long distances without relying on dedicated, active satellite relays or vulnerable ground infrastructure.

The recent proliferation of thousands of satellites in Low Earth Orbit (LEO), such as those in the Starlink, OneWeb, and Kuiper constellations, has filled the near-Earth space with a dense and predictable cloud of metallic objects. While these satellites are designed for active communication with their proprietary ground stations, their physical bodies (e.g., chassis, solar panels) are also reflective to radio frequencies. To date, these objects have not been systematically utilized as passive reflectors for unrelated, third-party communication links. The technical challenge lies in the extreme complexity of predicting the precise trajectory of thousands of rapidly moving objects and calculating

the exact moment when one will be perfectly positioned to create a specular reflection between two specific points on Earth.

2 Summary of the Invention

The present invention provides a method and system to overcome these limitations by treating the existing cloud of orbiting objects as a dynamic, statistically predictable field of potential signal reflectors.

A primary object of the invention is to provide a system that calculates a spatio-temporal intersection point—a specific point in space at a specific future time—where an orbiting object will be positioned to reflect a signal from a transmitter to a receiver.

Another object of the invention is to provide a method for retrieving real-time or near-real-time orbital parameter data (e.g., Two-Line Element sets) for a large number of orbiting objects.

A further object of the invention is to provide a computational engine that predicts the future trajectories of said objects based on this data and identifies a candidate object and an associated event time for a communication link.

The system comprises a transmitting station, a receiving station, and a central computation engine. The computation engine continuously ingests orbital data, calculates potential reflection geometries, and provides the transmitting station with precise firing solutions (i.e., target azimuth, elevation, and timing) to establish a link.

In one embodiment, the transmitting and receiving stations are equipped with movable, steerable antennas. In another embodiment, the stations are equipped with a plurality of fixed antennas pointing towards cardinal directions, and the system waits for an orbiting object to align with one of the pre-defined transmission paths.

The invention enables a form of communication that is highly secure, as the specific satellite and timing used for any given transmission are ephemeral and selected from thousands of possibilities, making interception extraordinarily difficult. It also provides a potential communication method for remote areas, disaster zones, or any scenario where traditional infrastructure is unavailable.

3 Brief Description of the Drawings

FIG. 1 is a conceptual diagram illustrating the overall system, showing a ground transmitter, a target receiver, and a non-cooperative satellite acting as a passive reflector.

FIG. 2 is a geometric diagram illustrating the reflection principle, showing the incident and reflected signal paths and the object's position relative to the transmitter and receiver.

FIG. 3 is a flowchart of the computational method for identifying a viable communication opportunity.

FIG. 4 is a diagram contrasting the operational modes for a system with a movable antenna versus a system with fixed cardinal antennas.

4 Detailed Description of the Invention

The core of the invention is a computational system that solves a complex, four-dimensional (3D space + time) kinematic problem. The system is designed to find a future moment in time when a non-cooperative orbiting object will pass through a specific point in space that satisfies the geometric conditions for specular reflection between a ground transmitter and a ground receiver.

4.1 System Architecture

The system consists of three main components:

1. **Data Ingestion Module:** This module is responsible for periodically fetching orbital parameter data for a large plurality of orbiting objects. A primary source for this data is the public Two-Line Element (TLE) set repository maintained by CelesTrak. This data provides the necessary parameters to model the orbit of each object. The data is cached locally to minimize external dependencies.

2. **Computational Engine:** This is the central processing unit of the system. It takes as input the locations of the sender and receiver and the orbital data from the ingestion module. It performs a search over a future time window to find a viable communication opportunity. Due to the computational intensity, the engine is preferably implemented as a distributed, massively parallel system. A cloud-based task queue architecture is used, where hundreds or thousands of small computational tasks are created, each responsible for evaluating a specific, narrow slice of the future time window.

3. **Transceiver Stations:** The physical ground stations. Each station includes a directional antenna (or array of antennas) and a radio transceiver. The transmitting station is controlled by the computational engine, which dictates the precise moment to send a directed signal.

4.2 Computational Method

The computational method differs based on the type of antenna system employed.

4.2.1 Movable Antenna Mode

This mode offers the highest flexibility and is suitable for dynamic or mobile communication needs (e.g., ship-to-shore).

1. **Initialization:** The system receives the geographic coordinates (latitude, longitude) of the sender (S) and receiver (R).

2. **Time-Slicing:** The system defines a future search window (e.g., the next 10 minutes) and divides it into discrete time steps, t_i (e.g., every 30 seconds). The following steps are performed for each t_i.

3. **Ephemeris Calculation:** Using a celestial mechanics library such as Skyfield, the system calculates the Earth-Centered, Earth-Fixed (ECEF) position vectors for the sender, $P_S(t_i)$, and the receiver, $P_R(t_i)$.

4. **Satellite Position Prediction:** For each of the thousands of satellites in the TLE dataset, the system predicts its ECEF position vector, $P_{sat}(t_i)$.

5. **Geometric Reflection Check:** For a signal to reflect from the satellite to the receiver, the satellite's position must satisfy the law of reflection. The system models the Earth's surface at the satellite's ground track as the reflection plane. It calculates the surface normal vector, N_{sat}, at the satellite's position. The vector from the sender to the satellite, $V_{in} = P_S - P_{sat}$, and the vector from the receiver to the satellite, $V_{out} = P_R - P_{sat}$, are calculated. A valid reflection occurs if the angle of incidence equals the angle of reflection with respect to the normal vector. This is computationally verified by checking if:

$$\frac{V_{in} \cdot N_{sat}}{\|V_{in}\|} \approx \frac{V_{out} \cdot N_{sat}}{\|V_{out}\|}$$

   within a predefined tolerance.

6. **Line-of-Sight Check:** The system verifies that the satellite is above a minimum elevation angle (e.g., 10 degrees) from the horizon for both the sender and the receiver to ensure a clear line of sight.

7. **Firing Solution:** If a satellite and time step (sat_j, t_i) satisfy these conditions, a firing solution is generated. The system outputs the satellite's name, the precise time of the event, and the required azimuth and elevation for the sender's antenna to target the satellite at that instant. A countdown is initiated for the transmission.

A key optimization involves filtering the satellites based on distance and kinematics. For transmissions between different hemispheres, only satellites moving in the correct direction (i.e., ascending or descending orbit) are considered.

4.2.2 Fixed Cardinal Antenna Mode

This mode is suitable for communication between fixed installations, where movable antennas are impractical. Each station is assumed to have an array of antennas pointing to fixed cardinal directions (e.g., North, East, South, West) at a fixed elevation.

1. **Initialization:** The system receives the sender and receiver coordinates and precalculates the fixed line-of-sight (LOS) vectors for each antenna in the ECEF coordinate system.

2. **Plausible Path Pruning:** The system determines which pairs of sender-receiver antennas are geometrically plausible. For example, for a sender in Texas to communicate with a receiver in New York, a plausible path might involve the sender's North-facing antenna and the receiver's South-facing antenna. This pruning dramatically reduces the search space.

3. **Keyhole Search:** For each plausible antenna pair and for each time step t_i, the system iterates through the satellites. It checks if a satellite's predicted position $P_{sat}(t_i)$ falls within a narrow conical "keyhole" defined by the sender's antenna LOS vector.

4. **Reflection Vector Calculation:** If a satellite is found in the sender's keyhole, the system calculates the ideal reflection vector based on the surface normal at the satellite's position (N_{sat}) and the incoming signal vector (which is aligned with the sender's LOS).

5. **Recipient Alignment Check:** The system then checks if this calculated reflection vector aligns with the actual vector from the satellite to the receiver ($P_R - P_{sat}$). It also verifies that this vector falls within the recipient antenna's corresponding keyhole.

6. **Solution Generation:** If all conditions are met, a communication opportunity is confirmed. Since the antennas are fixed, the firing solution only consists of the precise transmission time.

4.3 System Implementation and Embodiment

A prototype of the system has been implemented as a distributed web application using the Python programming language, the Flask web framework, and the Skyfield library. The computational engine is deployed on a cloud platform (e.g., Google App Engine) and uses a task queue (e.g., Google Cloud Tasks) to parallelize the search across hundreds of virtual workers. A NoSQL database (e.g., Google Firestore) is used to manage the state of the search and store the results.

The system intelligently selects which satellite constellations to search based on the Haversine distance between the two ground points. For distances greater than a threshold (e.g., 5000 km), higher-altitude MEO and GEO satellites are prioritized, as LEO satellites would be below the horizon. For shorter distances, the denser LEO constellations are used.

This embodiment demonstrates that the complex calculations required to find a viable bounce opportunity can be performed within a practical timeframe (typically under 30 seconds), making the system feasible for real-world application. To further enable one of ordinary skill in the art to practice the invention, a complete, working source code implementation of this embodiment is provided in Appendix A.

A Appendix A: Source Code for Prototype Implementation

```
# -*- coding: utf-8 -*-
"""
Finds the first opportunity to bounce a signal off a supported LEO, MEO,
or GEO satellite between two points on Earth.

Intended Use: Forwards cat videos (or Kats, or Lyons videos) to a friend on another
continent with the highest possible latency, using a multi-billion dollar satellite
constellation as a mirror. Please do not use for practical purposes.
Or for communicating with aliens. They have better FTL methods.

Date: June 25, 2025
"""
from flask import Flask, render_template, request, jsonify, Response
from datetime import datetime, timedelta, timezone
import numpy as np
import requests
import io
import os
import time
import traceback
import uuid
import json

from skyfield.api import load, wgs84
from google.cloud import firestore, tasks_v2

# --- App Initialization ---
app = Flask(__name__)

# --- Initialize clients only if in production ---
db = None
tasks_client = None
if os.environ.get('GAE_ENV', '').startswith('standard'):
    try:
        db = firestore.Client(database='bounce')
        tasks_client = tasks_v2.CloudTasksClient()
        print("Successfully connected to Firestore and Cloud Tasks.")
    except Exception as e:
        print(f"CRITICAL ERROR: Failed to initialize Google Cloud clients: {e}")

```

```
# --- Environment Config & Constants ---
GCP_PROJECT_ID = os.environ.get('GCP_PROJECT_ID')
GCP_QUEUE_NAME = os.environ.get('GCP_QUEUE_NAME')
GCP_QUEUE_LOCATION = os.environ.get('GCP_QUEUE_LOCATION')
WORKER_URL = os.environ.get('WORKER_URL')

timescale = load.timescale()
TLE_CACHE_DIR = "/tmp/tle_cache/"
TLE_CACHE_DURATION_SEC = 7200 # Cache for 2 hours
TLE_URLS_LEO = {
    "LEO": {
        "Starlink": "https://celestrak.org/NORAD/elements/gp.php?GROUP=starlink&FORMAT=tle",
        "OneWeb": "https://celestrak.org/NORAD/elements/gp.php?GROUP=oneweb&FORMAT=tle",
        "Kuiper": "https://celestrak.org/NORAD/elements/gp.php?GROUP=kuiper&FORMAT=tle",
    }
}
TLE_URLS_MEO_GEO = {
    "MEO/GEO": {
        "GPS": "https://celestrak.org/NORAD/elements/gp.php?GROUP=gps-ops&FORMAT=tle",
        "Galileo": "https://celestrak.org/NORAD/elements/gp.php?GROUP=galileo&FORMAT=tle",
        "Geostationary": "https://celestrak.org/NORAD/elements/gp.php?GROUP=geo&FORMAT=tle",
    }
}
LEO_MAX_DISTANCE_KM = 5000
MOVABLE_SEARCH_DURATION_SEC = 600
FIXED_SEARCH_DURATION_SEC = 3600
SEARCH_STEP_SEC = 30
BROAD_PHASE_RADIUS_M = 3000 * 1000
MOVABLE_MIN_ELEVATION_DEG = 10.0
MOVABLE_TOLERANCE_DEG = 10.0
FIXED_ELEVATION_DEG = 25.0
KEYHOLE_TOLERANCE_DEG = 2.5

ANTENNA_CONFIGS = [
    {'direction': 'N', 'azimuth': 0.0, 'elevation': FIXED_ELEVATION_DEG},
    {'direction': 'E', 'azimuth': 90.0, 'elevation': FIXED_ELEVATION_DEG},
    {'direction': 'S', 'azimuth': 180.0, 'elevation': FIXED_ELEVATION_DEG},
    {'direction': 'W', 'azimuth': 270.0, 'elevation': FIXED_ELEVATION_DEG},
]

# --- Helper Functions ---
```

```
def get_tle_data(url_map):
    if not os.path.exists(TLE_CACHE_DIR): os.makedirs(TLE_CACHE_DIR)
    full_tle_string = ""
    headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.36'}
    for group_name, urls in url_map.items():
        for name, url in urls.items():
            cache_file_path = os.path.join(TLE_CACHE_DIR, f"{name.replace('/', '_')}.txt")
            if os.path.exists(cache_file_path) and (time.time() - os.path.getmtime(cache_file_path)) < TLE_CACHE_DURATION_SEC:
                with open(cache_file_path, 'r') as f: full_tle_string += f.read() + "\n"
                continue
            try:
                response = requests.get(url, timeout=15, headers=headers)
                response.raise_for_status()
                tle_data = response.text
                with open(cache_file_path, 'w') as f: f.write(tle_data)
                full_tle_string += tle_data + "\n"
            except requests.exceptions.RequestException as e:
                print(f"Warning: Failed to fetch from {url}: {e}")
                if os.path.exists(cache_file_path):
                    with open(cache_file_path, 'r') as f: full_tle_string += f.read() + "\n"
    return full_tle_string if full_tle_string else None

def parse_tle_string(tle_data):
    try:
        temp_tle_filename = '/tmp/temp_tle_file.txt'
        with open(temp_tle_filename, 'w') as f: f.write(tle_data)
        satellites = load.tle_file(temp_tle_filename)
        name_map = {sat.model.satnum: sat.name for sat in satellites}
        return satellites, name_map
    finally:
        if os.path.exists(temp_tle_filename): os.remove(temp_tle_filename)

def haversine_distance(lat1, lon1, lat2, lon2):
    R = 6371
    dLat, dLon, lat1_rad, lat2_rad = map(np.radians, [lat2 - lat1, lon2 - lon1, lat1, lat2])
    a = np.sin(dLat/2)**2 + np.cos(lat1_rad) * np.cos(lat2_rad) * np.sin(dLon/2)**2
```

```
    c = 2 * np.arctan2(np.sqrt(a), np.sqrt(1-a))
    return R * c

def got_los_vector(lat, lon, az, el):
    lat_rad, lon_rad, el_rad, az_rad = map(np.radians, [lat, lon, el, az])
    s_el, c_el, s_az, c_az = np.sin(el_rad), np.cos(el_rad), np.sin(az_rad), np.cos(az_rad)
    enu_vec = np.array([s_az * c_el, c_az * c_el, s_el])
    rot_matrix = np.array([
        [-np.sin(lon_rad), -np.cos(lon_rad)*np.sin(lat_rad), np.cos(lon_rad)*np.cos(lat_rad)],
        [np.cos(lon_rad), -np.sin(lon_rad)*np.sin(lat_rad), np.sin(lon_rad)*np.cos(lat_rad)],
        [0, np.cos(lat_rad), np.sin(lat_rad)]
    ])
    return rot_matrix.dot(enu_vec)

# --- THE TWO SEARCH ENGINES (Now check only a single timestamp) ---
def find_movable_bounce_at_time(satellites, name_map, sender, recipient, t):
    sender_geo, recipient_geo = wgs84.latlon(sender['lat'], sender['lon']), wgs84.latlon(recipient['lat'], recipient['lon'])
    use_kinematic_filter = abs(recipient['lat'] - sender['lat']) >= 1.0
    if use_kinematic_filter: required_direction_is_ascending = (recipient['lat'] > sender['lat'])

    p_sender_obj, p_recipient_obj = sender_geo.at(t), recipient_geo.at(t)
    p_sender_m, p_recipient_m = p_sender_obj.position.m, p_recipient_obj.position.m
    midpoint_ecef = (p_sender_m + p_recipient_m) / 2

    initial_candidates = satellites
    if use_kinematic_filter: initial_candidates = [sat for sat in satellites if (sat.at(t).velocity.km_per_s[2] > 0) == required_direction_is_ascending]

    nearby_candidates = [(sat, sat.at(t)) for sat in initial_candidates if np.linalg.norm(sat.at(t).position.m - midpoint_ecef) < BROAD_PHASE_RADIUS_M]

    for sat, p_sat_obj in nearby_candidates:
        p_sat_m = p_sat_obj.position.m
        v_normal, v_in, v_out = p_sat_m/np.linalg.norm(p_sat_m), p_sender_m - p_sat_m, p_recipient_m - p_sat_m
        v_in_norm, v_out_norm = v_in / np.linalg.norm(v_in), v_out / np.linalg.norm(v_out)
```

```
            if not np.isclose(np.dot(v_in_norm, v_normal), np.dot(
                v_out_norm, v_normal), atol=MOVABLE_TOLERANCE_DEG/10):
                 continue
            elevation_sender, azimuth_sender, _ = (p_sat_obj -
                p_sender_obj).altaz()
            if elevation_sender.degrees < MOVABLE_MIN_ELEVATION_DEG:
                continue
            elevation_recipient, azimuth_recipient, _ = (p_sat_obj -
                p_recipient_obj).altaz()
            if elevation_recipient.degrees <
                MOVABLE_MIN_ELEVATION_DEG: continue
            sat_geo = wgs84.geographic_position_of(p_sat_obj)
            return {
                'satellite_name': name_map.get(sat.model.satnum, f"ID
                    {sat.model.satnum}"), 'satellite_norad_id': sat.
                    model.satnum,
                'event_timestamp_utc': t.utc_iso(),
                'sender_location': [sender['lat'], sender['lon']], '
                    recipient_location': [recipient['lat'], recipient[
                    'lon']],
                'satellite_ground_track': [sat_geo.latitude.degrees,
                    sat_geo.longitude.degrees], 'satellite_altitude_km
                    ': sat_geo.elevation.km,
                'elevation_sender': elevation_sender.degrees, '
                    azimuth_sender': azimuth_sender.degrees,
                'elevation_recipient': elevation_recipient.degrees, '
                    azimuth_recipient': azimuth_recipient.degrees,
                'stats': {'total': len(satellites), 'kinematic': len(
                    initial_candidates), 'nearby': len(
                    nearby_candidates)}
            }
    return None

def find_fixed_bounce_at_time(satellites, name_map, sender,
    recipient, t, plausible_paths):
    sender_geo, recipient_geo = wgs84.latlon(sender['lat'],
        sender['lon']), wgs84.latlon(recipient['lat'], recipient['
        lon'])
    p_sender_m, p_recipient_m = sender_geo.at(t).position.m,
        recipient_geo.at(t).position.m

    for sender_antenna, recipient_antenna in plausible_paths:
        v_los_sender = get_los_vector(sender['lat'], sender['lon'
            ], sender_antenna['azimuth'], sender_antenna['
            elevation'])
        v_los_recipient = get_los_vector(recipient['lat'],
            recipient['lon'], recipient_antenna['azimuth'],
            recipient_antenna['elevation'])
        for sat in satellites:
            p_sat_obj, p_sat_m = sat.at(t), sat.at(t).position.m
            v_sender_to_sat_norm = (p_sat_m - p_sender_m) / np.
```

```
                    linalg.norm(p_sat_m - p_sender_m)
178                 angle_diff = np.degrees(np.arccos(np.clip(np.dot(
                        v_sender_to_sat_norm, v_los_sender), -1.0, 1.0)))
179                 if angle_diff < KEYHOLE_TOLERANCE_DEG:
180                     v_normal = p_sat_m / np.linalg.norm(p_sat_m)
181                     v_in_norm = -v_sender_to_sat_norm
182                     v_reflection = v_in_norm - 2 * np.dot(v_in_norm,
                            v_normal) * v_normal
183                     v_sat_to_recipient_norm = (p_recipient_m -
                            p_sat_m) / np.linalg.norm(p_recipient_m -
                            p_sat_m)
184                     reflection_angle_diff = np.degrees(np.arccos(np.
                            clip(np.dot(v_reflection,
                            v_sat_to_recipient_norm), -1.0, 1.0)))
185                     if reflection_angle_diff < KEYHOLE_TOLERANCE_DEG:
186                         angle_diff_recipient = np.degrees(np.arccos(
                                np.clip(np.dot(-v_sat_to_recipient_norm,
                                v_los_recipient), -1.0, 1.0)))
187                         if angle_diff_recipient <
                                KEYHOLE_TOLERANCE_DEG:
188                             sat_geo = wgs84.geographic_position_of(
                                    p_sat_obj)
189                             return {
190                                 'sender_antenna_direction':
                                        sender_antenna['direction'], '
                                        recipient_antenna_direction':
                                        recipient_antenna['direction'],
191                                 'satellite_name': name_map.get(sat.
                                        model.satnum, f"ID {sat.model.
                                        satnum}"),
192                                 'satellite_norad_id': sat.model.
                                        satnum, 'event_timestamp_utc': t.
                                        utc_iso(),
193                                 'satellite_ground_track': [sat_geo.
                                        latitude.degrees, sat_geo.
                                        longitude.degrees], '
                                        satellite_altitude_km': sat_geo.
                                        elevation.km,
194                                 'stats': {}
195                             }
196     return None
197
198 # --- MAIN ROUTER and Asynchronous Handlers ---
199 @app.route('/')
200 def index():
201     return render_template('index.html')
202
203 @app.route('/find-bounce', methods=['POST'])
204 def find_bounce_router():
205     data = request.json
206     try:
```

```
        sender, recipient, mode = data.get('sender'), data.get('recipient'), data.get('mode', 'movable')
        if not all([sender, recipient, mode]): return jsonify({'status': 'ERROR', 'message': 'Invalid input'}), 400

        search_id = str(uuid.uuid4())
        search_duration = MOVABLE_SEARCH_DURATION_SEC if mode == 'movable' else FIXED_SEARCH_DURATION_SEC
        total_steps = search_duration // SEARCH_STEP_SEC

        db.collection('hyper_searches').document(search_id).set({
            'status': 'PENDING', 'created_at': firestore.SERVER_TIMESTAMP,
            'results': [], 'steps_completed': 0, 'total_steps': total_steps, 'mode': mode
        })

        parent = tasks_client.queue_path(GCP_PROJECT_ID, GCP_QUEUE_LOCATION, GCP_QUEUE_NAME)
        t_search_start_iso = datetime.now(timezone.utc).isoformat()

        for t_offset in range(0, search_duration, SEARCH_STEP_SEC):
            task_payload = {
                'search_id': search_id, 'sender': sender, 'recipient': recipient,
                'mode': mode, 't_search_start_iso': t_search_start_iso, 't_offset': t_offset
            }
            task = {'http_request': {'http_method': 'POST', 'url': f"{WORKER_URL}/run-timeslice-worker", 'headers': {'Content-Type': 'application/json'}, 'body': json.dumps(task_payload).encode() }, 'dispatch_deadline': timedelta(minutes=10) }
            tasks_client.create_task(parent=parent, task=task)

        print(f"Successfully created {total_steps} parallel worker tasks for search {search_id}.")
        return jsonify({'status': 'Search initiated', 'search_id': search_id})
    except Exception as e:
        traceback.print_exc()
        return jsonify({'status': 'ERROR', 'message': str(e)}), 500

@app.route('/run-timeslice-worker', methods=['POST'])
def run_timeslice_worker():
    data = request.json
    search_id, sender, recipient, mode, t_search_start_iso, t_offset = \
```

```
        data.get('search_id'), data.get('sender'), data.get('
            recipient'), \
        data.get('mode'), data.get('t_search_start_iso'), data.
            get('t_offset')

    doc_ref = db.collection('hyper_searches').document(search_id)
    try:
        ground_dist_km = haversine_distance(sender['lat'], sender
            ['lon'], recipient['lat'], recipient['lon'])
        tle_urls_to_fetch = TLE_URLS_MEO_GEO if ground_dist_km >
            LEO_MAX_DISTANCE_KM else TLE_URLS_LEO
        constellation_name = list(tle_urls_to_fetch.keys())[0]
        tle_data = get_tle_data(tle_urls_to_fetch)

        if tle_data:
            satellites, name_map = parse_tle_string(tle_data)
            t_search_start = datetime.fromisoformat(
                t_search_start_iso)
            current_time = t_search_start + timedelta(seconds=
                t_offset)
            t = timescale.from_datetime(current_time)

            result = None
            if mode == 'movable':
                result = find_movable_bounce_at_time(satellites,
                    name_map, sender, recipient, t)
            else: # fixed
                plausible_sender_dirs, plausible_recipient_dirs =
                     set(), set()
                if recipient['lat'] > sender['lat']:
                    plausible_sender_dirs.add('N')
                if recipient['lat'] < sender['lat']:
                    plausible_sender_dirs.add('S')
                if recipient['lon'] > sender['lon']:
                    plausible_sender_dirs.add('E')
                if recipient['lon'] < sender['lon']:
                    plausible_sender_dirs.add('W')
                if not plausible_sender_dirs:
                    plausible_sender_dirs = {'N','S','E','W'}
                if sender['lat'] > recipient['lat']:
                    plausible_recipient_dirs.add('S')
                if sender['lat'] < recipient['lat']:
                    plausible_recipient_dirs.add('N')
                if sender['lon'] > recipient['lon']:
                    plausible_recipient_dirs.add('W')
                if sender['lon'] < recipient['lon']:
                    plausible_recipient_dirs.add('E')
                if not plausible_recipient_dirs:
                    plausible_recipient_dirs = {'N','S','E','W'}
                plausible_paths = [(s_ant, r_ant) for s_ant in
                    ANTENNA_CONFIGS if s_ant['direction'] in
```

```
                        plausible_sender_dirs for r_ant in
                        ANTENNA_CONFIGS if r_ant['direction'] in
                        plausible_recipient_dirs]

                    result = find_fixed_bounce_at_time(satellites,
                        name_map, sender, recipient, t,
                        plausible_paths)

                if result:
                    result['constellation_searched'] =
                        constellation_name
                    @firestore.transactional
                    def update_in_transaction(transaction, doc_ref,
                        result):
                        snapshot = doc_ref.get(transaction=
                            transaction)
                        if snapshot.get('status') == 'PENDING':
                            transaction.update(doc_ref, {'results':
                                firestore.ArrayUnion([result]), '
                                status': 'SUCCESS'})
                    transaction = db.transaction()
                    update_in_transaction(transaction, doc_ref,
                        result)
        except Exception as e:
            print(f"Worker for timestamp offset {t_offset} failed: {e
                }")
        finally:
            @firestore.transactional
            def increment_counter(transaction, doc_ref):
                transaction.update(doc_ref, {'steps_completed':
                    firestore.Increment(1)})
            transaction = db.transaction()
            increment_counter(transaction, doc_ref)
        return 'OK', 200

@app.route('/get-search-status/<search_id>') # Renamed to be
    generic
def get_hyper_search_status(search_id):
    if db is None: return jsonify({'status': 'ERROR', 'message':
        'Database not configured'}), 500
    doc_ref = db.collection('hyper_searches').document(search_id)
    doc = doc_ref.get()
    if not doc.exists: return jsonify({'status': 'INVALID_ID'}),
        404
    data = doc.to_dict()
    if data.get('status') == 'SUCCESS':
        return jsonify({'status': 'SUCCESS', 'result': data['
            results'][0], 'mode': data.get('mode')})
    completed = data.get('steps_completed', 0)
    total = data.get('total_steps', 120)
    if completed >= total:
```

```
306             duration = MOVABLE_SEARCH_DURATION_SEC if data.get('mode'
                    ) == 'movable' else FIXED_SEARCH_DURATION_SEC
307             return jsonify({'status': 'COMPLETE_NO_RESULT', '
                    search_duration_min': int(duration / 60)})
308         return jsonify({'status': 'PENDING', 'steps_completed':
                completed, 'total_paths': total})
309
310 @app.route('/_ah/warmup')
311 def warmup():
312     print("Warmup request received. Initializing instance and pre
            -caching TLE data.")
313     get_tle_data(TLE_URLS_LEO)
314     get_tle_data(TLE_URLS_MEO_GEO)
315     return '', 200, {}
```

B Claims

What is claimed is:

1. A method for establishing a communication link, the method comprising the steps of:

   (a) Determining a first geographic location of a transmitting station and a second geographic location of a receiving station.

   (b) Retrieving orbital parameter data for a plurality of non-cooperative orbiting objects.

   (c) Executing instructions on a processor to perform a computational search, said search comprising:

      i. Predicting a future trajectory for each of the plurality of orbiting objects based on the retrieved orbital parameter data.

      ii. Identifying a future time and a future spatial position of a candidate orbiting object from the plurality, wherein said future spatial position satisfies a geometric condition for specular reflection of a signal from the transmitting station to the receiving station.

      iii. Generating a firing solution comprising at least the identified future time.

   (d) Transmitting a directed signal from the transmitting station at the identified future time towards the identified future spatial position, whereby the signal reflects from the candidate orbiting object and is received by the receiving station.

2. The method of claim 1, wherein the transmitting station comprises a movable antenna, and the firing solution further comprises an azimuth and an elevation for directing the movable antenna towards the future spatial position.

3. The method of claim 1, wherein the transmitting station comprises a plurality of antennas with fixed orientations, and wherein the computational search identifies a candidate orbiting object whose future trajectory aligns with the pre-determined orientation of one of the plurality of antennas.

4. The method of claim 1, wherein the step of identifying the future spatial position further comprises calculating an angle of incidence of a vector from the transmitting station to the object and an angle of reflection of a vector from the object to the receiving station, relative to a surface normal vector at the object's position, and confirming they are equal within a specified tolerance.

5. The method of claim 1, wherein the computational search is performed by a distributed system comprising a plurality of workers, wherein each worker is assigned to evaluate a distinct future time interval.

6. The method of claim 1, further comprising a step of calculating a distance between the first and second geographic locations and selecting a subset of the plurality of orbiting objects to search based on said distance, wherein lower-altitude objects are selected for shorter distances and higher-altitude objects are selected for longer distances.

7. A system for establishing a communication link, the system comprising:

   (a) A data ingestion module configured to retrieve orbital parameter data for a plurality of non-cooperative orbiting objects from a remote data source.

   (b) A computational engine communicatively coupled to the data ingestion module, the engine comprising one or more processors configured to:

      i. Receive a first geographic location of a transmitting station and a second geographic location of a receiving station.

      ii. Predict future trajectories for the plurality of orbiting objects based on the retrieved data.

      iii. Identify a candidate orbiting object, a future time, and a future spatial position where the object will satisfy a geometric condition for specular reflection of a signal from the transmitting station to the receiving station.

      iv. Generate a firing solution comprising at least the identified future time.

   (c) A transmitting station configured to receive the firing solution from the computational engine and to transmit a directed radio signal at the identified future time towards the identified future spatial position.

8. The system of claim 7, wherein the transmitting station comprises a movable antenna, and wherein the firing solution further comprises an azimuth and an elevation for aiming the antenna.

9. The system of claim 7, wherein the computational engine is further configured to filter the plurality of orbiting objects based on a kinematic property, such that only objects moving in a direction from the first geographic location towards the second geographic location are considered in the search.

10. The system of claim 7, wherein the non-cooperative orbiting objects are commercial communication satellites whose primary communication functions are not utilized by the system.

~~Drawings~~



~~Figure 1: A conceptual diagram illustrating the overall system, showing a ground transmitter, a target receiver, and a non-cooperative satellite acting as a passive reflector.~~

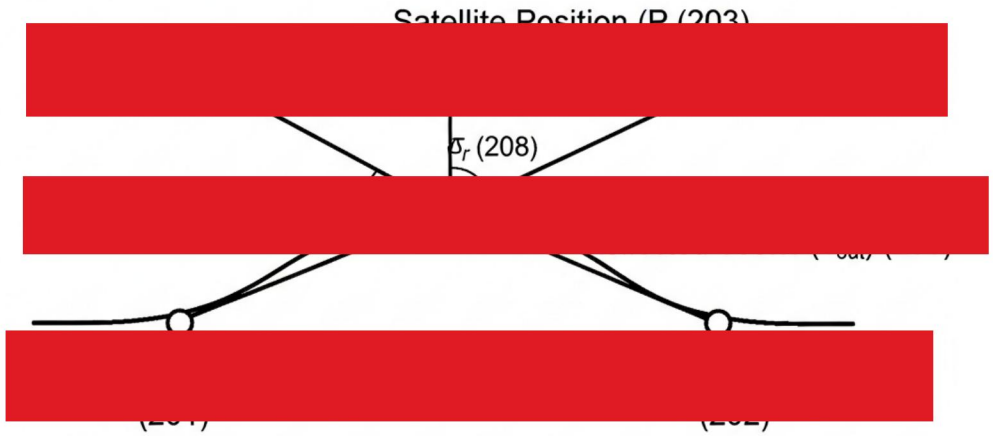


Figure 2: A geometric diagram illustrating the reflection principle, showing the incident and reflected signal paths and the object's position relative to the transmitter and receiver.

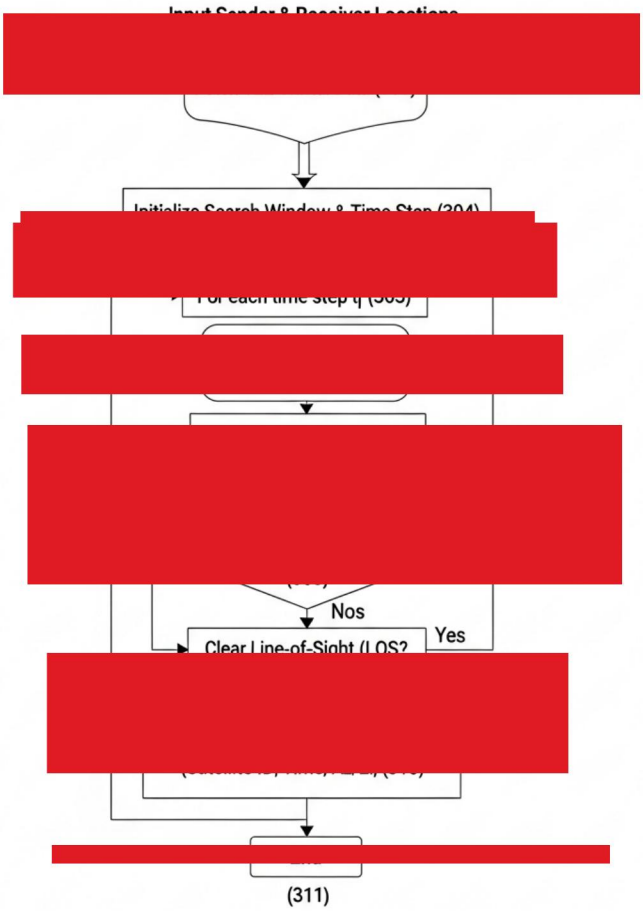


Figure 3: A flowchart of the computational method for identifying a viable communication opportunity.

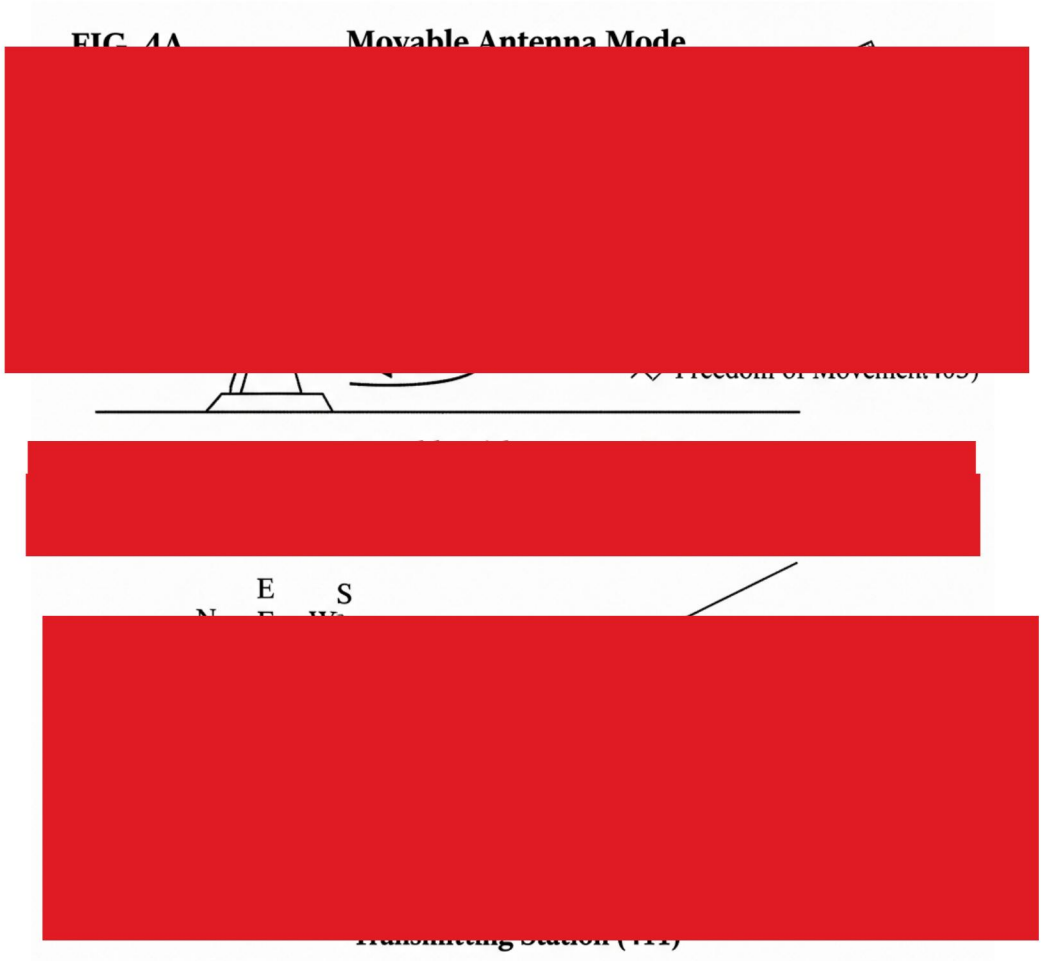


Figure 4: A diagram contrasting the operational modes for a system with a movable antenna versus a system with fixed cardinal antennas.